SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 15

Introduction to Software Design

Design Concepts, Abstraction, Modularity, Cohesion & Coupling

Student Book • Module III: Software Design

Learning focus
Understand how requirements are transformed into a structured software solution, and learn the design principles that make software easier to understand, test, change, and maintain.

Running case study: College Event Management System (CEMS)

Designed for classroom projection, discussion, practical application, and revision.

1. Learning Outcomes

By the end of this session, you should be able to:

• Explain the purpose of software design and its relationship to requirements.

• Distinguish high-level architectural design from detailed design.

• Explain abstraction, separation of concerns, information hiding, and stepwise refinement.

• Describe modularity and the principles of high cohesion and low coupling.

• Compare common forms of cohesion and coupling using simple examples.

• Apply design concepts to a College Event Management System.

Core idea: software design turns an understanding of what a system must do into a plan for how its parts will work together. Good design makes required behaviour easier to implement, test, understand, and change.

2. What Is Software Design?

Software design is the activity of defining the system's structure, components, interfaces, data organisation, and interactions so that the requirements can be implemented. Design sits between requirements engineering and coding.

Stage	Main question	Typical output
Requirements	What problem must the system solve?	Approved requirements, use cases, constraints, acceptance criteria
Design	How should the solution be organised to meet those requirements?	Architecture, modules, interfaces, data models, UML diagrams, design decisions
Implementation	How will the design be expressed in code or configuration?	Source code, database schema, integrations, build artefacts
Testing	Does the implementation satisfy the requirements and design expectations?	Test results, defects, evidence, acceptance decisions

Design is not decoration or only the layout of screens. It includes the organisation of responsibilities, data, interfaces, security boundaries, failure handling, and interactions between components.

3. Why Design Before Implementation?

• Complexity control: split a large problem into parts that can be understood.

• Consistency: establish common structures, interfaces, and conventions.

• Risk reduction: explore difficult integrations and quality requirements before full implementation.

• Testability: make responsibilities and interfaces clear enough to test independently.

• Maintainability: isolate changes so one feature does not require rewriting unrelated parts.

• Communication: help developers, testers, analysts, and stakeholders discuss the proposed solution.

Example: if event registration, event publishing, email notification, and reporting are mixed into one large component, a change to notification logic may break registration. Separating responsibilities helps the team reason about and test each area independently.

4. High-Level and Detailed Design

High-level design (HLD)	Detailed design (LLD)
Describes the major components and how they interact.	Describes the internal logic and structure of components.
Focuses on architecture, deployment, major data stores, external services, and system boundaries.	Focuses on classes, methods, algorithms, validation rules, data fields, and error handling.
Example: browser UI → application/API layer → database → email service.	Example: RegistrationService checks eligibility, capacity, duplicate registration, saves the result, and returns a status.

The boundary between HLD and LLD varies by organisation. Both should remain consistent with requirements and quality constraints.

5. Core Design Concept: Abstraction

Abstraction means concentrating on essential behaviour while temporarily hiding unnecessary detail. It helps people reason about a system at the right level.

Example

A student uses a “Register” button. The student needs to understand the action and result; they do not need to know the database transaction, capacity-locking logic, or notification API details. Those details matter to the implementation, but they are hidden behind the user-facing operation.

Levels of abstraction

• Business abstraction: “A student registers for an event.”

• System abstraction: “The registration service validates eligibility and capacity.”

• Implementation detail: “The service performs a transaction and writes a registration record.”

Good abstraction does not mean ignoring important details. It means revealing the details relevant to the current question and keeping other complexity behind a clear boundary.

6. Separation of Concerns

Separation of concerns means organising software so different responsibilities are handled in distinct places. A concern might be user interaction, business rules, data access, authentication, or notifications.

Concern	Example responsibility in CEMS
Presentation	Display event information, forms, validation messages, and registration status.
Business logic	Check eligibility, capacity, duplicate registration, and event rules.
Data access	Read and save event and registration records.
Identity and access	Identify users and enforce roles and permissions.
Notifications	Send confirmation or event-change messages when required.

If a single module handles all these concerns, it becomes harder to test and change. Separation does not require a particular framework; it is a design principle that can be applied in many technologies.

7. Information Hiding

Information hiding means keeping internal design decisions private behind a clear interface. Other components should depend on what a module promises to do, not on its internal implementation.

> **Example:** a RegistrationService exposes an operation such as registerStudent(studentId, eventId). Other modules do not need to know how the service checks capacity or persists the record. If the internal database structure changes but the service contract remains stable, fewer parts of the application need to change.

Benefits

• Reduces the ripple effect of internal changes.

• Prevents other modules from depending on implementation details.

• Supports independent testing and clearer responsibilities.

• Makes it easier to replace an internal mechanism without changing every caller.

8. Stepwise Refinement

Stepwise refinement means starting with a broad task and progressively decomposing it into more detailed steps until each part is sufficiently clear to implement.

Level	Registration example
Goal	Register a student for an event.
Subtasks	Validate user; find event; check eligibility; check duplicate; check capacity; save registration; return outcome.
More detail	If user is unauthenticated, request sign-in. If event is full, reject. If already registered, show existing status. Otherwise save a confirmed registration.
Implementation detail	Define service methods, data constraints, transaction behaviour, error codes, and automated tests.

Refinement reveals missing requirements and design decisions. If the team cannot explain a branch or expected result, it may need to revisit requirements before coding.

9. Modularity

Modularity is the practice of dividing a system into modules with well-defined responsibilities and interfaces. A module may be a function, class, package, service, library, or larger subsystem, depending on the architecture.

Example module breakdown

Module	Primary responsibility	Possible interface
Event Catalogue	Find and display published events.	listPublishedEvents(), getEventDetails(eventId)
Registration	Create and inspect registrations; enforce capacity rules.	registerStudent(), getRegistrationStatus()
Event Administration	Create, update, and publish events for authorised users.	createEvent(), updateEvent(), publishEvent()
Identity / Access	Identify users and check permissions.	authenticate(), hasRole()
Notification	Send approved notifications.	sendRegistrationConfirmation(), sendEventUpdate()

These are conceptual modules, not a mandate to build five separate deployable services. A small application might implement them as packages or classes within one deployment.

10. Cohesion: How Closely Related Are a Module's Responsibilities?

Cohesion describes how strongly the responsibilities inside one module belong together. In general, **high cohesion is desirable**: a module should have a clear, focused purpose.

Cohesion type	Description	Illustrative example
Coincidental	Unrelated tasks are grouped together.	One Utility module calculates capacity, formats emails, and changes user roles without a common purpose.
Logical	Similar categories of tasks are grouped, selected by a flag.	One module handles all kinds of input, with branches for keyboard, file, and network input.
Temporal	Tasks are grouped because they occur at the same time.	A startup routine loads configuration, sends a welcome email, and cleans unrelated temporary files.
Procedural	Tasks are grouped because they follow a particular sequence.	A module runs steps in order, though the steps may have weakly related purposes.
Communicational	Operations work on the same data set.	A module reads and updates event information for related operations.
Sequential	One operation's output becomes the next operation's input.	Validate registration data, then use the validated data to save the registration.
Functional	All elements contribute to one well-defined task.	A CapacityValidator determines whether an event can accept another registration.

Textbooks sometimes use slightly different names or groupings for cohesion categories. The practical design goal remains to keep each module's purpose focused and understandable.

11. Coupling: How Dependent Are Modules on Each Other?

Coupling describes the degree of dependency between modules. In general, **low coupling is desirable**: modules interact through small, stable, explicit interfaces rather than relying on each other's internal details.

Coupling type	Description	Illustrative example
Content	One module directly uses or changes another module's internals.	Registration code modifies private fields inside the Event Catalogue module.
Common	Modules share global data or shared mutable state.	Several modules change the same global currentEvent variable.
External	Modules depend on an externally imposed format, protocol, or device interface.	Several modules independently parse the same third-party response format.
Control	One module tells another which internal path to execute using control flags.	A service calls processRegistration(mode=4), where numeric modes choose hidden behaviour.
Stamp / data-structure	A whole record or object is passed when only part is needed.	A notification module receives an entire Student object to use only the email address.
Data	Modules exchange only the data needed through explicit parameters.	NotificationService receives recipientEmail and message.
Message	Modules communicate through messages or contracts with limited shared implementation knowledge.	A notification request is published using a defined message contract.

As with cohesion, terminology can vary by textbook. The useful principle is to avoid unnecessary dependency and keep interfaces small, clear, and stable.

12. Cohesion and Coupling Together

Cohesion looks inside a module; coupling looks at relationships between modules. A good design aims for high cohesion within modules and low coupling between them.

Design	Likely effect
One giant module that handles UI, database, registration, email, and reports	Low clarity and difficult testing; unrelated changes may affect each other.
Small modules with focused responsibilities and stable interfaces	Clearer ownership, easier testing, and more localised changes.
Too many tiny modules with excessive communication	Unnecessary complexity and overhead; modularity should be purposeful.
Modules that share internal data structures directly	Changes to one module's internals can break other modules.

The goal is not to create the maximum number of modules. The goal is to create meaningful boundaries that help the system meet its requirements and quality goals.

13. Case Study: Design CEMS Registration

Suppose CEMS initially has one large component called EventManager that displays pages, checks permissions, validates capacity, writes database records, sends emails, and builds reports.

Problems with the initial design

• Many responsibilities are mixed together, making the component difficult to understand.

• Testing registration rules may require the UI and database to be running.

• A change to email delivery may affect registration code.

• Other components may rely on internal data structures, creating high coupling.

A more focused conceptual design

Student UI → Registration Service → Event Repository / Registration Repository. The Registration Service uses an Access Control interface and asks a Notification Service to send messages after the registration outcome is known.

The service owns the registration workflow. Repositories own data access. The UI presents results. The notification module handles message delivery. Each module exposes a clear contract, and internal details remain hidden.

Questions to evaluate the design

• Does each module have a clear, coherent responsibility?

• Can the registration rules be tested without testing the entire UI?

• Can the notification mechanism change without rewriting capacity logic?

• Are permission checks enforced in the appropriate trusted layer?

• Are error conditions and transaction boundaries clear?

14. Common Design Mistakes

• Designing screens before understanding the underlying business rules.

• Creating a giant module that owns unrelated responsibilities.

• Splitting the application into many modules without clear boundaries.

• Allowing modules to change one another's internal data directly.

• Passing large objects when a small, explicit interface is enough.

• Using vague interfaces or hidden control flags that callers do not understand.

• Treating a conceptual module diagram as a final deployment architecture without considering scale, operations, and team needs.

15. Classroom Activity

Activity: Improve the design

Scenario: a single EventManager component displays events, creates registrations, checks capacity, saves records, sends email, and creates reports.

• Identify at least five distinct concerns in the component.

• Propose a modular breakdown with one clear responsibility per module.

• Write one example interface or operation for each module.

• Identify one example of high cohesion and one example of low cohesion.

• Identify two examples of high coupling and propose lower-coupling alternatives.

• Explain how your design makes one change easier to implement and test.

> **Expected deliverable:** a module table, a simple component sketch, and a short explanation of cohesion and coupling decisions.

16. Knowledge Check

1. What is software design?

Defining structure, components, interfaces, data organisation, and interactions to satisfy requirements.

2. What is abstraction?

Focusing on essential behaviour while hiding unnecessary detail at the current level.

3. What is information hiding?

Keeping internal implementation details behind a clear, stable interface.

4. What is modularity?

Dividing a system into modules with defined responsibilities and interfaces.

5. What is high cohesion?

A module's responsibilities are closely related and support one focused purpose.

6. What is low coupling?

Modules have limited, explicit dependencies on each other.

7. Which is generally preferred: high cohesion or low cohesion?

High cohesion.

8. Which is generally preferred: high coupling or low coupling?

Low coupling.

9. Does modularity always mean separate deployable services?

No. Modules may be functions, classes, packages, or subsystems within one application.

10. Why is a stable interface valuable?

It allows internal implementation to change with fewer effects on other modules.

17. Glossary

Term	Meaning
Software design	Planning the structure and interactions of a software solution.
Abstraction	Showing essential behaviour while hiding unnecessary detail.
Separation of concerns	Assigning different responsibilities to distinct parts of a system.
Information hiding	Keeping internal details behind a defined interface.
Stepwise refinement	Breaking a broad task into progressively more detailed steps.
Modularity	Organising software into parts with clear responsibilities.
Cohesion	How closely related the responsibilities inside a module are.
Coupling	How strongly one module depends on other modules.
Interface	A defined way for a module or system to interact with another.
Refactoring	Improving internal structure without intentionally changing external behaviour.

18. Homework and Deliverables

• Choose CEMS or another approved software system.

• Identify at least six responsibilities and group them into conceptual modules.

• For each module, write its responsibility and one example interface.

• Explain how your design aims for high cohesion and low coupling.

• Give two examples of information hiding and two examples of separation of concerns.

• Draw a simple module interaction diagram using paper or a tool such as diagrams.net.

• Write a short paragraph explaining how your design supports testability and maintainability.

Next session: Software Architecture and Architectural Design—layers, components, architectural styles, and choosing an architecture for a system.